

LINUX ER

The Emergency-Room Cheat Sheet

Treat every sick server like a patient. Find the failing organ, run the right command, and remember it forever with hooks you will never forget.

LOAD · CPU · MEM · DISK · NETWORK · SECURITY · LOGS

Curated by Shaam Natheshwar · DevOps / Cloud Engineer

Swipe →

0 ADMISSION

How to read this sheet in 30 seconds

Most cheat sheets are alphabetical lists. Real incidents are not alphabetical. When a server is sick you never ask "what does *awk* do?" You ask **"which part is hurting?"** So this sheet is built like an emergency room: every page is an organ, every organ has its vital signs, and every command has a memory hook so it sticks.

	ORGAN	WHAT YOU GET	THE QUESTION IT ANSWERS	PAGE
01	**VITAL SIGNS**	The 60-second triage. Run this before anything else.	*"Is it alive, and where does it hurt?"*	3
02	**FILESYSTEM**	The filesystem as a city map, and the boot chain.	*"Where does this thing live?"*	4
03	**CPU**	CPU, processes, signals, jobs, priority, limits.	*"Who is eating the CPU?"*	5
04	**MEM + DISK**	RAM, swap, OOM, disks, mounts, LVM.	*"Out of memory or out of space?"*	6
05	**NETWORK**	The 7-rung ping ladder, ports, DNS, firewalls.	*"Which hop is broken?"*	7
06	**SECURITY I**	Permissions, special bits, SELinux, the 5 checks.	*"Why is access denied?"*	8
07	**SECURITY II**	Users, groups, sudo, SSH keys and tunnels.	*"Who is allowed in, and how?"*	9
08	**LOGS + SERVICES**	Logs, systemd, cron, packages.	*"What happened, and how do I treat it?"*	10
09	**TOOLS I**	find, grep, globbing, regex, redirection.	*"How do I search and route output?"*	11
10	**TOOLS II**	awk, sed, sort, tar, links, rsync, one-liners.	*"How do I slice and move data?"*	12
11	**REFLEXES**	Shortcuts, exit codes, bash, vim.	*"How do I work faster?"*	13
12	**TRIAGE BOARD**	Symptom, first command, likely cause, 10-second drills.	*"Something broke. Now what?"*	14
13	**HOOK VAULT**	Every memory trick on one page.	*"Revise in 5 minutes."*	15

THREE CALLOUTS YOU WILL SEE ON EVERY PAGE

MEMORY HOOK
A trick to remember a command, flag or number: an acronym, a silly sentence, a picture. Example: **SUSH** = Sort, Uniq, Sort, Head.

SCAR
A mistake that really bites in production, usually learned at 2 a.m. Example: `usermod -G` without `-a` wipes every other group.

PRO MOVE
A habit senior engineers use that saves minutes. Example: `namei -l /path` shows exactly which parent directory is blocking you.

HOW TO USE IT: THE 3-PASS METHOD

PASS 1
Read only the HOOKS
Flip to page 15 and read the Hook Vault once. Five minutes. You now have the skeleton.

PASS 2
Cover and recall
Cover the right-hand column of any table. Say the command out loud from the description. Then flip it.

PASS 3
Break it on purpose
In a throwaway VM: fill a disk, kill a service, bind a busy port. Fix it using only page 14.

DISTRO TAGS

Ubuntu / Debian: `apt`, `dpkg`, `ufw`, `netplan`, `/var/log/syslog`, `/var/log/auth.log`, group `sudo`, AppArmor.

RHEL / Rocky / Alma / Fedora: `dnf`, `rpm`, `firewalld`, `nmcli`, `/var/log/messages`, `/var/log/secure`, group `wheel`, SELinux.

NOTE
Commands here are safe to read and run unless a **SCAR** says otherwise. Anything that changes a system (`rm`, `chmod -R`, `usermod`, `systemctl stop`) deserves a second look on production.

OR LEARN IT IN 7 DAYS: ONE ORGAN A DAY

DAY 1	DAY 2	DAY 3	DAY 4	DAY 5	DAY 6	DAY 7
Vitals + Filesystem p3 · p4	**CPU + MEM** p5 · p6	**Network** p7	**Security I + II** p8 · p9	**Logs + Services** p10	**Tools I + II** p11 · p12	**Reflexes, Board, Vault** p13 to p15

1 VITAL SIGNS

The 60-second triage: run it in this order, every single time

#	ORGAN	COMMAND	HEALTHY LOOKS LIKE	RED FLAG	GO TO
1	LOAD	`uptime`	Load averages at or below your core count (`nproc`).	15-min load above core count. Or an uptime of minutes: it rebooted.	p5
2	CPU	`top`	Mostly `id` (idle). No single process pinned at 100%.	High `wa` = disk. High `st` = noisy neighbour VM. Z = zombies.	p5
3	MEM	`free -h`	`available` is a healthy slice of `total`. Little swap used.	`available` under 10% and swap growing. OOM-killer is next.	p6
4	DISK	`df -hT ; df -i`	Use% under 85 for blocks AND inodes.	100% on any filesystem. Check inodes too, not just space.	p6
5	NETWORK	`ss -tulpn`	Your service is LISTENing on the right port and address.	Missing listener, or bound to 127.0.0.1 only.	p7
6	LOGS	`journalctl -p err -b -n 30`	A few harmless lines.	OOM, segfault, "Failed", I/O error, disk full, auth floods.	p10
7	KERNEL LOG	`sudo dmesg -T | tail -20`	Quiet.	`Killed process`, `EXT4-fs error`, link down, hardware faults.	p6

MEMORY HOOK

Read the order as one breath: **UP · TOP · FREE · DF · SS · JOURNAL · DMESG**. It also walks the system top to bottom: load, cpu, mem, disk, network, logs, kernel log. Never skip ahead to Google. The first failing organ is your lead.

ONE-SHOT TRIAGE: PASTE INTO ~/.bashrc, THEN JUST TYPE `er`

```
er() { uptime; echo; free -h; echo; df -hT -x tmpfs -x devtmpfs; echo; df -i -x tmpfs -x devtmpfs | head -8; echo; ss -tulpn | head -15; echo; journalctl -p err -b -n 15 --no-pager; }
```

DECODE THE CPU LINE IN top

%Cpu(s): 3.1 us, 1.2 sy, 0.0 ni, 94.8 id, 0.6 wa, 0.0 hi, 0.2 si, 0.1 st

us	sy	ni	id	wa	hi	si	st
user	**system**	**nice**	**idle**	**I/O wait**	**hard irq**	**soft irq**	**steal**
your apps	kernel work	low-priority jobs	doing nothing	WAITING ON DISK	hardware signals	kernel deferred	STOLEN BY HYPERVISOR

LOAD AVERAGE: THE BANK COUNTER

Load = people **at the counter or in line**. Cores = tellers. On 4 cores, load 4.0 means every teller is busy and nobody waits. Load 8.0 means a queue twice as long as your staff.

READ THE 3 NUMBERS	MEANS
`1.0 5.0 9.0`	1 min < 5 min < 15 min: **recovering**
`9.0 5.0 1.0`	1 min > 5 min > 15 min: **getting worse**
`9.0 9.0 9.0`	Steady pressure for 15 minutes: **sustained**

SCAR

High load with a **low** CPU% is a storage problem, not a CPU problem. Linux counts processes stuck waiting on disk (state `D`) as load. Check `wa` and run `iostat -xz 1`.

WHAT EACH RESOURCE SHOULD TELL YOU

`nproc`	Core count: your load yardstick
`top` then press `1`	Per-core view. One core pinned = a single-threaded hog
`top` then press `M` / `P`	Sort by Memory / by CPU
`free -h`	Look at `available`, not `free`
`vmstat 1 5`	`si/so` above 0 for long = swapping. `b` = blocked on I/O
`iostat -xz 1`	`%util` near 100 and big `await` = disk is the bottleneck

GOING DEEPER: THE USE METHOD, FOR EVERY RESOURCE

Utilization · Saturation · Errors

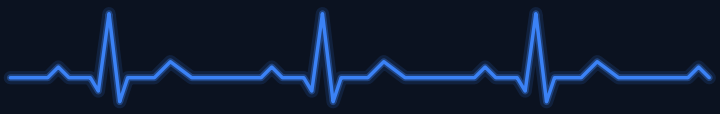
RESOURCE	UTILIZATION: how busy?	SATURATION: how much is waiting?	ERRORS: what is failing?
CPU	`top` (us + sy vs id)	`uptime` load vs `nproc`; `vmstat 1` column r	`dmesg -T` (machine-check, throttling)
Memory	`free -h` (available)	`vmstat 1` si/so; swap growing	`dmesg -T | grep -i oom`
Disk	`iostat -xz 1` (%util)	`iostat -xz 1` (await, aqu-sz)	`dmesg -T` I/O errors; `smartctl -a`
Network	`sar -n DEV 1` (rx/tx)	`ss -s`; `nstat` retransmits	`ip -s link` (errors, drops)

MEMORY HOOK

USE it: per resource, ask **U**tilization (how busy?), **S**aturation (work queuing?), **E**rrors (failing?). Errors are quick to check and most often skipped.

2 FILESYSTEM

The filesystem as a city, and how the machine wakes up



THE ROOT · CONFIG · BOOT · MOUNTS · PROGRAMS · DATA · VIRTUAL (lives in RAM, 0 bytes)

/ — *The planet*

Top of the tree. Everything hangs from here. Not the same as /root!

/etc — *Town Hall*

Config files: the city's rules. Plain text. Back up before editing.

/boot — *Ignition gate*

Kernel, initramfs, GRUB. A full /boot breaks kernel updates.

/mnt /media — *Parking lot*

Mount points. /mnt = manual, /media = auto (USB sticks).

/bin /sbin — *Toolshed*

Core commands. sbin = sysadmin tools. Now links into /usr.

/usr — *Shopping district*

Installed programs, libraries, docs (Unix System Resources).

/lib — *Power plant*

Shared libraries (.so files) and kernel modules.

/opt — *Outlet mall*

Self-contained third-party apps in their own bundle.

/home — *Residential*

One house per user: /home/alice. ~ is the shortcut.

/root — *Mayor's house*

root's private home. It is NOT inside /home.

/var — *Warehouse*

Stuff that changes: logs, caches, queues, DBs. The usual disk-filler.

/tmp — *Dumpster*

Scratch space, wiped on reboot (often). Sticky bit: the t.

/proc — *CCTV room*

Live window into processes and the kernel, as files.

/sys — *Control room*

Hardware and driver knobs, exposed as files.

/dev — *The docks*

Devices as files: sda, null, zero, tty.

/run — *Whiteboard*

Runtime state (PID files, sockets). Cleared at boot.

FROM POWER BUTTON TO PROMPT

the italic line = where each stage breaks and what you will see

FIRMWARE	GRUB	KERNEL	SYSTEMD	TARGET
BIOS / UEFI	bootloader	+ initramfs	PID 1	multi-user / graphical
breaks as: No boot device	*breaks as: grub rescue>*	*breaks as: panic / dracut shell*	*breaks as: emergency mode*	*breaks as: unit failed*

WHERE DO I LOOK?

`/etc/<service>/`	A service's config
`/var/log/ · journalctl`	Logs, old school and systemd
`/etc/passwd /etc/group`	Users and groups (hashes live in `/etc/shadow`)
`/etc/fstab`	What mounts at boot
`/etc/hosts /etc/resolv.conf`	Static names and DNS servers
`/etc/ssh/sshd_config`	SSH server settings
`/etc/sudoers.d/`	Drop-in sudo rules (edit with `visudo -f`)
`/etc/systemd/system/`	YOUR unit files and overrides
`/usr/lib/systemd/system/`	The PACKAGE's unit files: do not edit
`/etc/cron.d /var/spool/cron`	System and per-user cron jobs
`/proc/cpuinfo /proc/meminfo`	CPU and RAM facts, live
`/proc/<PID>/fd /environ`	A process's open files and env vars

NAME DECODERS

MEMORY HOOK

/etc: it literally meant "et cetera". Read it as **E**ditable **T**ext **C**onfig. **/var**: it **VAR**ies. It grows. It fills your disk. **/usr**: Unix System Resources (originally "user"). **/opt**: **OPT**ional software. **/sbin**: **S**ysadmin bin. **/proc**: **PROC**esses. **/dev**: **DEV**ices.

DISK FULL? HUNT IN THIS ORDER

`/var/log`	log files that never rotated
`/var/lib/docker`	images, layers, stopped containers
`/var/cache`	package and app caches
`/tmp /home`	forgotten dumps, backups, ISOs
`lsof +L1`	deleted files still held open by a process

MEMORY HOOK

The boot chain: **Bring Great Knowledge, Start Today** = **B**IOS, **G**RUB, **K**ernel, **S**ystemd, **T**arget.

PRO MOVE

`systemctl get-default` shows your boot target.
`systemd-analyze blame` shows who made boot slow.

3 CPU

Processes, signals, jobs, priority



SEE WHAT IS RUNNING

`ps aux`	Every process with CPU/MEM. *aux* = All, User-oriented, eXtended
`ps -ef`	Every process, full format, shows PPID (parent)
`ps -eo pid,ppid,stat,%cpu,%mem,cmd --sort=-%cpu | head`	Custom columns, hogs first
`pgrep -fa nginx`	Find PIDs by name, show full command
`pstree -p`	The family tree of processes
`lsof -p PID`	Every file and socket a process has open
`lsof -i :8080`	Who owns port 8080
`strace -f -p PID`	Live system calls: what is it actually doing?
`cat /proc/PID/status`	Memory, threads, state, straight from the kernel

PROCESS STATES: THE 5 LETTERS IN STAT

R Running | S Sleeping | D Disk-wait | Z Zombie | T sTopped

MEMORY HOOK

Red Snails Drive Zippy Tractors = **R**unning, **S**leeping, **D**isk-wait, **Z**ombie, s**T**opped.

S	Normal. Waiting for work. Most processes live here.
D	Uninterruptible, usually I/O. **Unkillable**, even by `kill -9`. Many D = storage or NFS trouble.
Z	Dead, but its parent never collected the exit status. Fix or restart the **parent**.
T	Paused (Ctrl+Z or SIGSTOP). Resume with `fg` or `kill -CONT`.

"TOO MANY OPEN FILES"

`ulimit -n`	Open-file limit for this shell
`cat /proc/PID/limits`	The limits a running process actually has
`ls /proc/PID/fd | wc -l`	How many files it holds right now
`LimitNOFILE=65535`	In the unit file `[Service]` for services

MEMORY HOOK

A service ignores `/etc/security/limits.conf`? Of course: **systemd services take limits from the unit file**, not from PAM. Set `LimitNOFILE`, then `daemon-reload` and restart.

SIGNALS: HOW TO ASK A PROCESS TO STOP

`kill PID`	Sends 15 (TERM): "please finish up and exit"
`kill -1 PID`	HUP: reload config, no restart
`kill -2 PID`	INT: the same as Ctrl+C
`kill -9 PID`	KILL: instant death. No cleanup. Last resort
`kill -STOP / -CONT PID`	Pause / resume
`pkill -f pattern`	Kill by matching the command line
`killall nginx`	Kill every process with that exact name

MEMORY HOOK

15 asks, 9 kills. 1 = "hello again" (reload). **2** = Interrupt (Ctrl+C). **9** = nine lives gone. Escalate in order: `kill` then wait 10 seconds then `kill -9`.

JOBS AND SESSIONS

`cmd &`	Run in background
`Ctrl+Z then bg`	Suspend, then continue in background
`jobs ; fg %1`	List jobs; bring job 1 to front
`nohup cmd > out.log 2>&1 &`	Survives you logging out
`tmux new -s work`	Persistent session. Detach: Ctrl+b d
`tmux attach -t work`	Come back to it, even after SSH drops

PRIORITY

`nice -n 10 ./job.sh`	Start it politely
`renice -n 5 -p PID`	Change a running process

MEMORY HOOK

Nice = polite. Range is -20 to 19. A process with nice **19** is the most polite, so it gets the **least** CPU. Only root can go negative (rude, but fast).

SCAR

Never start with `kill -9`. The process gets no chance to flush buffers, remove lock files or close DB connections. That is how you get corrupt data and stale PID files.

PER-PROCESS LENSES (package: sysstat)

`mpstat -P ALL 1`	Every core, every second
`pidstat 1`	CPU per process, rolling
`pidstat -d 1`	Disk reads/writes per process
`iostat -xz 1`	Per-disk `%util` and `await`
`sar -n DEV 1`	Network throughput per interface

4 MEM + DISK

Memory, swap, OOM, disks, mounts, LVM

MEM: RAM

`free -h`	total / used / free / buff-cache / **available**
`vmstat 1 5`	`si so` = swap in/out. Sustained non-zero = pain
`ps aux --sort=-%mem | head`	Top memory eaters
`swapon --show`	Is swap on, and how big?
`grep -E 'MemAvail|SwapFree' /proc/meminfo`	The raw truth
`dmesg -T | grep -i 'killed process'`	Did the OOM-killer fire?

MEMORY HOOK

Cache is cash. Linux banks spare RAM as buff/cache and spends it back instantly when an app asks. A tiny `free` is normal. Judge by **available**.

SCAR

Do not run `echo 3 > /proc/sys/vm/drop_caches` "to free memory". It only makes the box slower while it re-reads everything. In containers, **exit code 137 = 128 + 9** = SIGKILL, very often the OOM-killer.

DISK: SPACE

`df -hT`	Free space per filesystem, with type
`df -i`	**Inodes**. Full inodes = "No space left" with free GBs
`du -xh --max-depth=1 /var | sort -h`	Biggest folders, ascending (the culprit is last)
`ncdu /`	Interactive disk explorer (if installed)
`lsblk -f`	Disks, partitions, filesystems, UUIDs, mounts
`findmnt`	The mount tree
`lsof +L1`	Deleted-but-open files still eating space

MEMORY HOOK

df = the fridge, du = who ate the food. `df` tells you WHETHER you are out of space. `du` tells you WHO took it.

MOUNTS AND fstab

`mount /dev/sdb1 /data`	Mount now
`umount /data`	Unmount. If "busy": `fuser -vm /data`
`blkid /dev/sdb1`	Get the UUID for fstab
`mount -a`	Test fstab **before** you reboot

```
UUID=1a2b-3c4d /data xfs defaults,nofail 0 2
```

MEMORY HOOK

The 6 fstab fields: **Dear Mom, Take Our Dog, Please** = **D**evice, **M**ount point, **T**ype, **O**ptions, **D**ump, **P**ass (fsck order).

SCAR

A typo in `/etc/fstab` can drop you into emergency mode on next boot. Always `mount -a` first, and add `nofail` to non-critical mounts.

NEW DISK IN 6 MOVES

`1 lsblk`	Find it (say /dev/sdb)
`2 fdisk /dev/sdb`	`n` new, `w` write the table
`3 mkfs.xfs /dev/sdb1`	or `mkfs.ext4`
`4 mkdir /data`	Make the mount point
`5 blkid ; vi /etc/fstab`	Add the UUID line
`6 mount -a ; df -h`	Prove it works

GROWING A FILESYSTEM

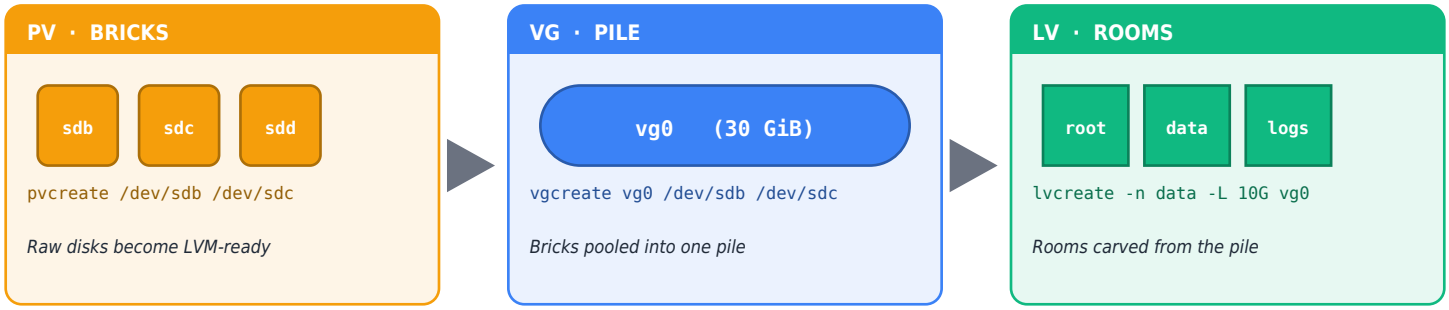
`lvextend -r -L +5G /dev/vg0/data`	Grow LV **and** the filesystem (`-r`)
`xfs_growfs /data`	XFS: grow online, pass the mount point
`resize2fs /dev/vg0/data`	ext4: grow, pass the device
`pvs ; vgs ; lvs`	Check each LVM layer

SCAR

XFS cannot shrink. Ever. Plan sizes up, or copy data out and rebuild.

LVM IN ONE PICTURE: BRICKS, PILE, ROOMS

PV → VG → LV (Physical, Volume group, Logical)



5 NETWORK

Networking: climb the ladder, stop at the first broken rung

THE 7-RUNG PING LADDER

the first rung that fails IS your bug

1 LINK	2 IP	3 GATEWAY	4 INTERNET	5 DNS	6 PORT	7 APP
`ip -br link`	`ip -br a`	`ip r` → `ping GW`	`ping -c3 1.1.1.1`	`dig +short example.com`	`nc -zv host 443`	`curl -I https://host`
Is the NIC UP?	*Do I have an address?*	*Can I reach the router?*	*Can I leave the LAN?*	*Do names resolve?*	*Is the door open?*	*Does the app answer?*

MEMORY HOOK

Lazy Iguanas Gulp Ice-cream, Drooling Proudly Afterwards = **L**ink, **I**P, **G**ateway, **I**nternet, **D**NS, **P**ort, **A**pp. Ping works but the site does not? Jump to rungs 5 to 7. Name fails but the IP works? That is DNS, always.

LOOK

`ip -br a`	Interfaces and IPs, brief
`ip -br link`	Link state, brief
`ip r`	Routing table (default via ... = gateway)
`ip route get 8.8.8.8`	Which route would be used?
`ss -tulpn`	Who is LISTENing on what
`ss -s`	Socket summary
`lsof -i :8080`	Who owns that port

MEMORY HOOK

ss -tulpn = "TULiP-N": **T**CP, **U**DP, **L**istening, **P**rocess, **N**umeric. Replaces the old `netstat -tulpn`.

TEST

`ping -c3 host`	Reachable?
`mtr -rwc 10 host`	ping + traceroute, with loss per hop
`dig +short A host`	What IP does DNS say?
`getent hosts host`	Resolve **like apps do** (honours /etc/hosts)
`nc -zv host 443`	Is the port open?
`curl -I https://host`	Headers only: status code, redirects
`curl -so /dev/null -w '%{http_code} %{time_total}\n' URL`	Status + total time, nothing else
`curl --resolve h:443:IP https://h`	Test one backend, skip DNS
`tcpdump -i any -nn port 53 -c 20`	Watch 20 DNS packets

FIREWALLS

`ufw status ; ufw allow 22/tcp`	Ubuntu. `ufw enable` to turn on
`firewall-cmd --list-all`	RHEL: what is open now
`firewall-cmd --add-service=http --permanent`	RHEL: add rule, saved
`firewall-cmd --reload`	RHEL: **now** make saved rules live
`iptables -L -n -v ; nft list ruleset`	The engine underneath

SCAR

firewalld: without `--permanent` the rule vanishes on reload. With `--permanent` it is not live until `--reload`. You need both.

NETWORK CONFIG

`netplan try`	Ubuntu: apply YAML, auto-rollback in 120 s. Use over SSH, never `apply`
`nmcli con show`	RHEL: connections
`nmcli con mod eth0 ipv4.addresses 10.0.0.5/24`	RHEL: change, then `nmcli con up eth0`
`hostnamectl set-hostname web01`	Set the hostname

PORTS WORTH MEMORISING

22	SSH	53	DNS
80 / 443	HTTP / HTTPS	25	SMTP
3306	MySQL	5432	Postgres
6379	Redis	1521	Oracle

MEMORY HOOK

Postgres 5432 counts down 5-4-3-2. **Redis 6379** spells **M-E-R-Z** on a phone keypad (6=M, 3=E, 7=R, 9=Z). **443** = padlock.

DECODE THE ERROR MESSAGE

what the client says tells you what the network did

YOU SEE	IT MEANS	CHECK
Connection refused	Host reached, **nothing listening** (or a REJECT rule)	`ss -tulpn` on server: up? right port?
Connection timed out	Packets **silently dropped** on the way	Firewall, security group, route, IP
No route to host	No usable path, or firewall ICMP reject	`ip r`, `ip route get IP`, REJECT rules
Name or service not known	DNS failed. The network may be fine	`dig +short host`, `getent hosts host`

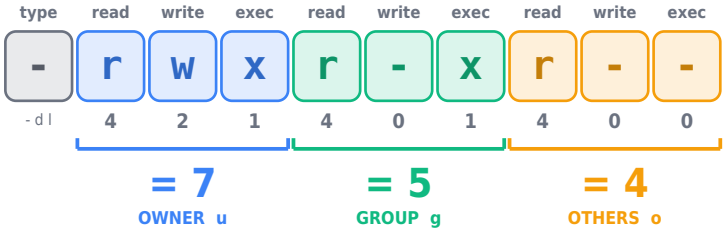
MEMORY HOOK

Refused = a door that said no. Timed out = a wall that said nothing.

6 SECURITY I

Permissions, special bits, SELinux, the 5 checks

```
$ ls -l app.sh    -rwxr-xr--  1  alice  dev  4096  Oct 15  app.sh
```



chmod 754 app.sh
7 · 5 · 4 = owner · group · others

On a directory: r = list the names, w = create or delete entries inside, x = enter it (cd). **x is the key to the door.**

MEMORY HOOK
4 - 2 - 1: **r**ead is 4, **w**rite is 2, e**x**ecute is 1. Add the ones you want per group: rwx = 7, r-x = 5, r-- = 4. **755** = owner rules, everyone else may look and run. **644** = owner writes, everyone reads. **600** = secrets only (SSH keys). **777** = never. Letters: **u g o a** = **U**ser, **G**roup, **O**thers, **A**ll.

CHANGE AND INSPECT

`chmod 640 f`	Numeric mode
`chmod u+x,g-w f`	Symbolic: add x for user, drop w for group
`chmod -R g+rwX dir`	Recursive; big X = x only on dirs and already-exec files
`chown alice:dev f`	Owner and group (add `-R` for trees)
`stat -c '%a %U:%G %n' f`	Mode, owner, group in one line
`namei -l /full/path/file`	Permissions of **every directory on the path**
`umask`	022 means files 644, dirs 755 by default
`setfacl -m u:bob:rw f ; getfacl f`	Per-user rights beyond owner/group
`lsattr f ; chattr +i f`	Immutable flag: even root cannot delete

SPECIAL BITS

4755	**setuid**: run as the file's owner (`passwd`)
2775	**setgid** on a dir: new files inherit the group
1777	**sticky** on a dir: only owners may delete (`/tmp` shows `t`)

MEMORY HOOK
The **t** in `/tmp`: **T**enants cannot throw out each other's stuff.

PERMISSION RECIPES

WHAT	FILES	DIRS
Web content	644	755
Private data / secrets	600	700
Team shared folder	664	2775
Scripts / binaries	755	n/a

FILE TYPE: THE FIRST CHARACTER

-	regular file	d	directory
l	symbolic link	c	character device
b	block device	s	socket

MEMORY HOOK
Daring Lions Chase Big Silly Pigs = d, l, c, b, s, p(ipe).

"PERMISSION DENIED"? RUN THE 5 CHECKS

`1 id`	**W**ho am I? Which groups do I really have?
`2 namei -l /full/path`	**P**ath: does every parent dir allow x?
`3 ls -Z ; getenforce`	**L**abel: is SELinux (or AppArmor) blocking?
`4 lsattr file`	**A**ttribute: is the immutable flag set?
`5 findmnt -T /path`	**M**ount: is it ro or noexec?

MEMORY HOOK
Wise Penguins Like Almond Milk = **W**ho, **P**ath, **L**abel, **A**ttribute, **M**ount. Stop at the first check that fails.

PRO MOVE
File mode looks right, yet access fails? It is almost always a **parent directory** (check 2). `namei -l` names the exact one missing x.

SELINUX (RHEL) / APPARMOR (UBUNTU)

`getenforce ; ls -Z ; ps -Z`	Mode and labels. The **Z** is the secret label
`ausearch -m avc -ts recent`	What SELinux just denied
`restorecon -Rv /path`	Reset labels to the default
`getsebool -a | grep httpd`	Feature switches (booleans)
`aa-status`	AppArmor profiles (Ubuntu)

SCAR
`setenforce 0` is a diagnosis, not a fix. If it works with SELinux off, find the missing label or boolean and turn enforcing back on.

SECURITY AUDIT IN 3 LINES

`find / -xdev -type f -perm -0002 -ls`	World-writable files
`find / -xdev -nouser -o -nogroup`	Files whose owner no longer exists
`find / -xdev -perm -4000 -type f`	Every setuid binary

PRO MOVE
Run the audit lines once on a fresh install and save the output. Next time, `diff` it. Anything new is worth a question.

7 SECURITY II

Users, groups, sudo, SSH keys and tunnels

USERS AND GROUPS

`useradd -m -s /bin/bash -G sudo alice`	Create with home, shell, extra group
`passwd alice ; passwd -l alice`	Set password / lock account
`usermod -aG docker alice`	Add to a group. **-a is vital**
`id alice ; groups alice`	UID, GID, memberships
`userdel -r alice`	Delete user and home
`chage -l alice`	Password aging and expiry
`w ; last ; lastb`	Who is on / past logins / failed logins
`getent passwd | awk -F: '$3==0'`	Every account with UID 0 (should be only root)

SCAR
`usermod -G docker alice` (no `-a`) **replaces** every group alice had, including `sudo`. Always `-aG`.

MEMORY HOOK
-aG = Always Group: **a**ppend, **G**roups. Admin group: **Ubuntu = sudo**, **RHEL = wheel** (the big wheel, the boss).

```
alice:x:1001:1001:Alice Doe:/home/alice:/bin/bash
```

MEMORY HOOK
The 7 fields of `/etc/passwd`: **Never Put Uncle's Greasy Cookies Here, Sir** = **N**ame, **P**assword (x), **U**ID, **G**ID, **C**omment, **H**ome, **S**hell. UID 0 = root. Under 1000 = system accounts.

SUDO

`visudo ; visudo -f /etc/sudoers.d/ops`	Edit sudo rules with syntax checking
`sudo -l`	What am I allowed to run?
`sudo -u postgres psql`	Run as another user
`sudo -i / su - alice`	Full login shell as root / as alice

```
alice ALL=(ALL) NOPASSWD: /usr/bin/systemctl
```

MEMORY HOOK
Read a sudoers line as **Who Where=(As whom) What**: alice, on ALL hosts, as ALL users, may run only `systemctl`, with no password.

SCAR
Never edit `/etc/sudoers` with a plain editor. One typo can lock out every admin. `visudo` validates first.

SHARED TEAM FOLDER IN 5 MOVES

`1 groupadd devs`	Create the group
`2 usermod -aG devs alice`	Add members (log out/in to apply)
`3 mkdir /srv/team`	Make the folder
`4 chgrp devs /srv/team`	Give it to the group
`5 chmod 2775 /srv/team`	2 = new files inherit the group

SSH

`ssh-keygen -t ed25519 -C me@work`	Make a key pair
`ssh-copy-id user@host`	Install your public key
`ssh -i key -p 2222 user@host`	Custom key and port
`ssh -J bastion user@private`	Hop through a jump host
`ssh -L 8080:localhost:80 user@host`	**L**ocal: pull a remote port to ME
`ssh -R 9000:localhost:3000 user@host`	**R**emote: push my port to THEM
`scp f user@host:/dir/`	Quick copy (rsync is smarter)
`sshd -t && systemctl reload ssh`	Test config, then reload (`sshd` on RHEL)

MEMORY HOOK
SSH is picky about privacy: **700 on ~/.ssh, 600 on keys and authorized_keys**. If a key "does nothing", check this first.

~/.ssh/config: TYPE LESS

```
Host prod
  HostName 10.0.0.12
  User ubuntu
  IdentityFile ~/.ssh/prod.pem
  ProxyJump bastion
```

PRO MOVE
Now `ssh prod` replaces a 60-character command, and `scp`, `rsync` and Ansible reuse the same alias.

HARDEN sshd_config

`PermitRootLogin no`	No direct root over SSH
`PasswordAuthentication no`	Keys only
`AllowUsers alice bob`	Whitelist who may log in
`MaxAuthTries 3`	Slow down guessing

SCAR
Keep your current SSH session open while you test changes in a **second** window. A bad config plus a closed session equals a locked door.

MEMORY HOOK
L = Local = bring it to me. R = Remote = send mine to them. Remember it from where the **port** ends up.

WHO IS KNOCKING?

`lastb | head`	Recent failed logins
`grep 'Failed password' /var/log/auth.log | awk '{print $(NF-3)}' | sort | uniq -c | sort -rn | head`	Top attacking IPs (RHEL: `/var/log/secure`)

PRO MOVE
Seeing a flood of failed logins? Move SSH to keys only, then add `fail2ban` to ban repeat offenders automatically.

8 LOGS + SERVICES

Logs, systemd, cron, packages

LOGS: JOURNAL

`journalctl -u nginx -f`	Follow one service live
`journalctl -u nginx --since '1 hour ago'`	Time-boxed
`journalctl -p err -b`	Errors this boot (`-b -1` = last boot)
`journalctl -xe`	End of log, with explanations
`journalctl -k`	Kernel messages only
`journalctl --disk-usage`	How big is it? `--vacuum-time=7d` trims
`tail -F /var/log/app.log`	Follow by name, survives rotation
`logrotate -d /etc/logrotate.conf`	Dry run: what would rotate?

MEMORY HOOK
-xe = "eXplain Errors": `x` adds explanations, `e` jumps to the end. Capital `-F` in tail = follow through rotation.

LOG FILE	UBUNTU	RHEL
System	`/var/log/syslog`	`/var/log/messages`
Logins / sudo	`/var/log/auth.log`	`/var/log/secure`
Cron	grep CRON in syslog	`/var/log/cron`
Package ops	`/var/log/apt/`	`/var/log/dnf.log`

systemd: THE SERVICE BOSS

`systemctl status svc`	State + last log lines
`systemctl start | stop | restart | reload svc`	Right now
`systemctl enable --now svc`	At boot **and** right now
`systemctl is-active svc ; is-enabled svc`	Script-friendly checks
`systemctl --failed`	Everything that is broken
`systemctl cat svc`	Show the effective unit file
`systemctl edit svc`	Safe override (drop-in)
`systemctl daemon-reload`	After editing any unit file
`timedatectl ; chronyc tracking`	Clock, timezone, NTP sync

MEMORY HOOK
start = now, enable = at boot, --now = both. Edited a unit? **daemon-reload first, restart second.**

A UNIT FILE IN 9 LINES

```
[Unit]
Description=My app
After=network.target

[Service]
User=app
ExecStart=/opt/app/run
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

MEMORY HOOK
Three sections: **Unit, Service, Install** = "You Serve It". Won't start? `journalctl -u svc -n 50`, then run `ExecStart` by hand as that user.

cron: THE ALARM CLOCK

small → big

MIN	HOUR	DOM	MON	DOW
*	*	*	*	*
0-59	0-23	1-31	1-12	0-7

MEMORY HOOK
Read it **small to big**: **Minute, Hour, Day, Month, Weekday**. Like a clock zooming out. `*/5` = every 5. `0 2 * * 1-5` = 02:00, Mon to Fri.

`crontab -e ; crontab -l`	Edit / list your jobs
`*/15 * * * * /opt/x.sh >> /var/log/x.log 2>&1`	Every 15 min, capture output
`@reboot /opt/start.sh`	Once at boot
`systemctl list-timers`	The systemd alternative

SCAR
Cron has a **tiny PATH** and no shell profile. Use full paths (`/usr/bin/python3`). A literal `%` must be written `\%`. Service is `cron` on Ubuntu, `crond` on RHEL.

PACKAGES: UBUNTU VS RHEL

TASK	APT / DPKG	DNF / RPM
Refresh index	`apt update`	`dnf makecache`
Upgrade all	`apt upgrade`	`dnf upgrade`
Install	`apt install p`	`dnf install p`
Remove	`apt remove p`	`dnf remove p`
Search	`apt search p`	`dnf search p`
Owner of file	`dpkg -S /path`	`rpm -qf /path`
Files of pkg	`dpkg -L p`	`rpm -ql p`
Installed list	`dpkg -l`	`rpm -qa`
Local file	`apt install ./x.deb`	`dnf install ./x.rpm`
Pin version	`apt-mark hold p`	`dnf versionlock add p`
Repo files	`sources.list.d/`	`yum.repos.d/`

MEMORY HOOK
update = the menu, upgrade = dinner. `apt update` only refreshes the list of what exists. Nothing is installed until `upgrade`.

PRO MOVE
Logs vanish after reboot? Make the journal persistent: `mkdir -p /var/log/journal && systemctl restart systemd-journald`.

SCAR
`reload` only works if the service defines it. When unsure use `systemctl reload-or-restart svc`.

9 TOOLS I

Find files, search text, route output

find: WHERE, WHAT, DO

`find /var/log -name '*.log'`	By name (quote the glob!)
`find . -type f -size +100M`	Files over 100 MB (`-type d` = dirs)
`find . -mtime +7`	Modified **more than** 7 days ago
`find . -mtime -1`	Modified **within** the last day
`find . -mmin -30`	Within the last 30 minutes
`find / -perm -4000 -type f 2>/dev/null`	Every setuid file (security audit)
`find . -user alice`	Owned by alice
`find . -name '*.tmp' -delete`	Find and delete
`find . -name '*.log' -exec gzip {} +`	Run a command on matches, batched
`find . -type f -print0 | xargs -0 cmd`	Safe with spaces in names

MEMORY HOOK

Minus = within, plus = older. `-mtime -1` is "less than 1 day ago". `-mtime +7` is "more than 7 days ago". Think of the number line: minus is closer to now.

SCAR

Always run the `find` **without** `-delete` first. Read the list. Then add `-delete`.

grep: THE TEXT HOUND

`grep -rniw 'error' /var/log/app`	Recursive, case-insens., line numbers, whole word
`grep -v '^#' file`	Hide comment lines
`grep -c 'ERROR' file`	Just count the matches
`grep -rl 'TODO' .`	Only the file names
`grep -E 'warn|fail' file`	Either word (extended regex)
`grep -A3 -B2 'panic' file`	3 lines After, 2 Before
`grep -o '[0-9.]\{7,15\}' file`	Print only the matching part

MEMORY HOOK

-v = inVert = the Villain filter (hides matches). **-A** After, **-B** Before, **-C** Context (both). **-r** Recursive, **-i** Ignore case, **-n** Number lines, **-l** List files, **-c** Count.

REGEX IN 8 SYMBOLS (grep -E)

`^ $`	Start / end of the line
`.`	Any one character
`* + ?`	0 or more / 1 or more / optional
`[abc] [0-9]`	One character from a set or range
`(a|b)`	Either a or b
`{3} {2,5}`	Exactly 3 / between 2 and 5

```
grep -E '^(ERROR|WARN)' app.log
```

MEMORY HOOK

^ is a cap on the START, $ is where you pay at the END.

REDIRECTION: THE THREE PIPES



Three pipes: In = 0, Out = 1, Err = 2 (I-O-E, count from zero)

`cmd > f`	Overwrite f with stdout
`cmd >> f`	Append to f
`cmd 2> err.log`	Only errors to a file
`cmd > all.log 2>&1`	Both into one file
`cmd &> all.log`	Bash shortcut for the line above
`cmd 2>/dev/null`	Silence errors (/dev/null = black hole)
`cmd < in.txt`	Feed a file as input
`cmd | tee -a out.log`	See it **and** save it
`cat <<'EOF' > f`	Here-doc: type text until a line `EOF`

MEMORY HOOK

One arrow = overwrite, two arrows = append (double the arrows, add more). `2>&1` reads "**send 2 to wherever 1 goes**". The `&` means "a file descriptor, not a file named 1". `tee` is shaped like a T-pipe: it splits the flow.

SCAR

Order matters: `cmd > f 2>&1` works. `cmd 2>&1 > f` does **not**: errors still hit the screen because 1 was copied **before** it moved to the file.

FINDING COMMANDS

`type cmd ; which cmd`	What will actually run? alias, function, or file
`whereis cmd`	Binary, source, man page
`command -v cmd`	Script-safe "does it exist?"
`man -k word ; cmd --help`	Search manuals / quick help
`echo $PATH | tr : '\n'`	One PATH entry per line

WILDCARDS AND BRACES

`*`	Any run of characters: `ls *.log`
`?`	Exactly one character: `file?.txt`
`[a-c]`	One character from the set
`{src,docs,tests}`	Brace expansion: `mkdir -p proj/{src,docs,tests}`
`~`	Your home directory

PRO MOVE

The **shell** expands wildcards before the command ever runs, so `rm *.log` really becomes `rm a.log b.log`. Preview any dangerous glob first: `echo rm *.log`.

10 TOOLS II

Slice text, pack files, link and sync

TEXT TOOLS

`cut -d: -f1,7 /etc/passwd`	Columns by delimiter: name and shell
`awk -F: '{print $1,$3}' f`	Print columns 1 and 3 (colon-separated)
`awk '$3 > 1000' /etc/passwd`	Filter rows by a column value
`awk '{s+=$1} END{print s}' f`	Sum a column
`sed 's/old/new/g' f`	Replace on every line
`sed -i.bak 's/a/b/' f`	Edit in place, keep a .bak
`sed -n '5,10p' f`	Print only lines 5 to 10
`sed '/^#/d' f`	Delete comment lines
`sort -k2 -nr f`	Sort by column 2, numeric, reversed
`uniq -c`	Count repeats (input **must be sorted**)
`tr 'a-z' 'A-Z'`	Translate characters
`wc -l f ; head -n5 ; tail -n5`	Count lines / first / last
`column -t f`	Align into neat columns
`diff -u a b`	What changed between two files

MEMORY HOOK

awk sees columns as **$1 $2 $3**, the whole line as `$0`, **N**umber of **F**ields as `NF`, line number as `NR`. `$NF` = the last column. **sed**: `s/old/new/g` = **S**wap, slash, old, slash, new, slash, **g**lobal.

ONE-LINERS WORTH STEALING

`du -sh * | sort -h | tail`	Biggest items in this folder
`find . -type f -printf '%s %p\n' | sort -nr | head`	Largest files below here
`ss -tan | awk 'NR>1{print $1}' | sort | uniq -c`	Connections by TCP state
`awk '{print $9}' access.log | sort | uniq -c | sort -rn`	HTTP status codes (nginx/apache)
`tail -f app.log | grep --line-buffered ERROR`	Live-filter a log
`watch -n2 'df -h /var'`	Re-run every 2 seconds
`for h in web1 web2; do ssh $h uptime; done`	Same command on many hosts
`history | awk '{print $2}' | sort | uniq -c | sort -rn | head`	Your most-used commands

SORT AND COMPARE

`sort -u f`	Sort and drop duplicates in one go
`sort -h`	Sort human sizes (`2K`, `10M`, `1G`)
`sort -t, -k3 -n f`	CSV: sort by the 3rd comma field
`comm -12 <(sort a) <(sort b)`	Lines present in **both** files
`diff -rq dirA dirB`	Which files differ between two folders

THE GREATEST ONE-LINER IN LINUX

```
awk '{print $1}' access.log | sort | uniq -c | sort -rn | head
```

MEMORY HOOK

S.U.S.H. = **S**ort, **U**niq -c, **S**ort -rn, **H**ead. Counts anything and shows the top 10: noisy IPs, popular URLs, repeated errors. Sort **first**, because `uniq` only merges neighbours.

ARCHIVES

`tar czf a.tgz dir/`	Create, gzip, to file
`tar xzf a.tgz -C /dest`	Extract into /dest
`tar tzf a.tgz`	Tell: list without extracting
`gzip f ; gunzip f.gz`	Compress one file
`zip -r a.zip dir/ ; unzip a.zip`	Zip for other platforms

MEMORY HOOK

Create Zee Files = czf, **eXtract Zee Files = xzf**, **Tell = tzf**. **f is always last** because it is followed by the file name.

LINKS

`ln -s /real/target linkname`	Symbolic (soft): a shortcut
`ln file hardname`	Hard: a second name for the same data
`ls -li ; readlink -f x`	Inodes / where does a link finally point

MEMORY HOOK

`ln -s` follows `cp` order: **source first, then the new name**. Soft link = shortcut, breaks if the target moves. Hard link = second name on the same inode; the data lives until the **last** name is removed.

COPY AND SYNC

`rsync -avz --delete -n src/ host:/dest/`	-n = dry run first
`rsync -avz --progress src/ dest/`	Real run, with progress
`cp -a src dst`	Copy preserving everything
`cp nginx.conf{,.bak}`	Brace trick = `cp nginx.conf nginx.conf.bak`

MEMORY HOOK

rsync -avz = **A**rchive (keep perms/times), **V**erbose, **Z**ip in transit. A trailing slash matters: `src/` = the **contents**, `src` = the folder itself.

VERIFY, INSPECT, SPLIT

`sha256sum f ; sha256sum -c SUMS`	Make / verify a checksum
`file x ; stat x`	What is it? Size, times, inode
`split -b 100M big part_`	Cut into 100 MB pieces
`cat part_* > big`	Glue them back together

PRO MOVE

Always verify a download or a transfer with `sha256sum`. Corruption is silent and always shows up at the worst time.

11 REFLEXES

Shell shortcuts, exit codes, bash, vim: work like muscle memory



KEYBOARD (bash)

`Ctrl+R`	Search history as you type (**R**ewind)
`Ctrl+A / Ctrl+E`	Jump to start / end of line
`Ctrl+W / Ctrl+U / Ctrl+K`	Delete word / to start / to end (**K**ill the rest)
`Alt+.`	Paste the last argument of the previous command
`Ctrl+L`	Clear screen
`Ctrl+C / Ctrl+D / Ctrl+Z`	Interrupt / end-of-input (logout) / suspend
`Ctrl+X Ctrl+E`	Open the line in `$EDITOR`
`!!`	Repeat the last command
`sudo !!`	Re-run it with sudo. "I said **please**."
`!$`	Last argument of the previous command
`^old^new`	Re-run the last command with a fix
`cd -`	Go back to the previous directory
`ls -lhtr`	Long, Human sizes, Time-sorted, Reversed

MEMORY HOOK
`ls -lhtr` = **Long, Human, Time, Reverse**: newest file lands at the **bottom**, right where your eyes already are.

EXIT CODES: echo $?

`0`	Success
`1`	General error
`2`	Misuse / bad arguments
`126`	Found, but **not executable** (`chmod +x`)
`127`	**Command not found** (typo or PATH)
`130`	Ctrl+C (128 + 2)
`137`	SIGKILL (128 + 9): OOM-killer or `kill -9`
`143`	SIGTERM (128 + 15): asked to stop

MEMORY HOOK
126 = can't Run, 127 = can't Find. Above 128 = killed by a signal: subtract 128 to get the signal number.

ALIASES WORTH STEALING (~/.bashrc)

`alias ll='ls -lahF'`	Detailed listing
`alias ports='ss -tulpn'`	Who listens where
`alias dfh='df -hT -x tmpfs -x devtmpfs'`	Clean disk view
`alias grep='grep --color=auto'`	Highlight matches
`source ~/.bashrc`	Reload without logging out

BASH SCRIPT SKELETON

```
#!/usr/bin/env bash
set -euo pipefail

name="${1:?usage: $0 NAME}"
trap 'echo failed at line $LINENO' ERR

if [[ -f "$name" ]]; then
  echo "exists"
fi
for f in *.log; do
  gzip "$f"
done
while read -r line; do
  echo "$line"
done < input.txt
```

MEMORY HOOK
`set -euo pipefail` = **three emergency brakes**: **e** stop on any error, **u** stop on an unset variable, **o pipefail** stop if any pipe stage fails.

VAR	MEANS	VAR	MEANS
`$0`	script name	`$?`	last exit code
`$1..$9`	arguments	`$$`	this PID
`$#`	arg count	`$!`	last bg PID
`"$@"`	all args (quote it)	`${v:-x}`	default if empty

TEST	TRUE WHEN	TEST	TRUE WHEN
`-f f`	is a file	`-d d`	is a directory
`-e p`	exists	`-z s`	string empty
`-eq -lt -gt`	numbers	`== !=`	strings

MEMORY HOOK
Numbers use letters, strings use symbols. `[ "$a" -eq 5 ]` for numbers, `[[ $s == yes ]]` for text. Mixing them up is the #1 beginner bug.

SCAR
`rm -rf "$DIR"/*` with `$DIR` empty becomes `rm -rf /*`. Use `"${DIR:?}"` so the script aborts instead.

vim SURVIVAL KIT

`i / Esc`	Insert mode / back to normal mode
`:w :q :wq ZZ`	Save / quit / both / both (fast)
`:q!`	Quit, throw changes away
`/text n N`	Search, next, previous
`dd yy p u`	Delete line / copy line / paste / undo
`:%s/old/new/g`	Replace in the whole file
`gg G :set nu`	Top / bottom / line numbers

MEMORY HOOK
Stuck in vim? **Esc, Esc, :q!** = "Escape the Escape". Then breathe.

12 TRIAGE BOARD

Symptom → first move → likely cause. Pin this one up.

SYMPTOM	FIRST MOVE	LIKELY CAUSE	PAGE
Server is slow	`uptime ; top`	Load > cores: CPU hog. High `wa`: disk. `si/so`: swapping. High `st`: noisy VM.	p3 · p5
No space left on device	`df -h ; df -i`	`/var/log`, docker, `/tmp`. Inodes full? Deleted-but-open files: `lsof +L1`.	p6
Service won't start	`systemctl status svc ; journalctl -u svc -n 50`	Bad config (`nginx -t`), port in use, wrong perms/user, missing env or file.	p10
Address already in use	`ss -tulpn | grep :PORT`	Another process owns the port. Stop it or change yours.	p7
Permission denied	`ls -l ; namei -l /path ; id`	Parent dir lacks `x`, wrong owner/group, SELinux (`ls -Z`), immutable bit (`lsattr`).	p8
Cannot SSH in	`ssh -vvv user@host ; nc -zv host 22`	Firewall / security group, sshd down, key perms (600), wrong user, `AllowUsers`.	p7 · p9
command not found	`type cmd ; echo $PATH`	Not installed, not on PATH, or a typo. Exit code 127.	p13
Process got killed	`dmesg -T | grep -i killed`	OOM-killer. Exit 137. Check `free -h` and per-process memory.	p6
Works locally, not from outside	`ss -tulpn ; firewall-cmd --list-all`	Bound to 127.0.0.1 only, host firewall, cloud security group, wrong port.	p7
Name does not resolve	`dig +short host ; getent hosts host`	Wrong `/etc/resolv.conf`, DNS server down, stale `/etc/hosts` entry.	p7
Cron job didn't run	`crontab -l ; grep CRON /var/log/syslog`	Minimal PATH, relative paths, unescaped `%`, no output capture, service down.	p10
Filesystem is read-only	`mount | grep ' ro,' ; dmesg | grep -i error`	FS errors or a failing disk. Unmount and `fsck`, then remount rw.	p6
Can't delete a file	`lsattr f ; ls -ld dir`	Immutable flag, no `w` on the **directory**, or the file is a mount point.	p8
High load, low CPU	`ps -eo state,pid,cmd | grep '^D'`	Processes stuck in D state: dead NFS, slow disk, storage array issues.	p5
Clock is wrong	`timedatectl ; chronyc tracking`	NTP not syncing. Breaks TLS, Kerberos, logs correlation and cron.	p10

THE 5 GOLDEN RULES OF THE ER

1 — Read the whole error
Out loud. The answer is usually in the sentence you skipped.

2 — Logs before Google
`journalctl -xe` first. Your box knows more than a forum.

3 — Change one thing
Then test. Two changes at once means no lessons learned.

4 — Back up before editing
`cp file{,.bak}`. Two seconds now, hours later.

5 — Reproduce, then fix
If you cannot trigger it, you cannot prove you fixed it.

10-SECOND DRILLS: COVER THE RIGHT SIDE, THEN ANSWER

comment your score

1. Disk shows 100%, but `du` finds nothing. Why?	Deleted file still held open: `lsof +L1`
2. A process in state D will not die with `kill -9`. Why?	Stuck in uninterruptible I/O (storage/NFS)
3. Which two characters send errors where stdout goes?	`2>&1` (and write it AFTER the `>`)
4. chmod mode for a private SSH key?	`600` (and `700` on ~/.ssh)
5. What does exit code 137 mean?	128 + 9 = SIGKILL, often the OOM-killer
6. Top 10 noisiest IPs in a log?	`... | sort | uniq -c | sort -rn | head` (SUSH)
7. Who owns port 8080?	`ss -tulpn | grep 8080` or `lsof -i :8080`
8. `usermod` without `-a` does what?	Replaces ALL the user's groups

13 HOOK VAULT

Every memory trick on one page: read it, then cover it



#01
Up · Top · Free · Df · Ss · Journal · Dmesg

The 60-second triage order. One breath, seven commands.

#02
4 - 2 - 1

r=4, w=2, x=1. Add them per group: rwx=7, r-x=5, r--=4.

#03
x = the key to the door

On a directory, x means you may enter it. r only lists the names.

#04
Czf · Xzf · Tzf

Create / eXtract / Tell, with Zee Files. f is always last.

#05
Dear Mom, Take Our Dog, Please

fstab: Device, Mount point, Type, Options, Dump, Pass.

#06
TULiP-N

ss -tulpn: TCP, UDP, Listening, Process, Numeric.

#07
S.U.S.H.

Sort | Uniq -c | Sort -rn | Head. Top-N of anything.

#08
Soft = shortcut, Hard = 2nd name

ln -s: source first, then new name. Same order as cp.

#09
df = fridge, du = who ate

df: am I out of space? du: who took it?

#10
Bricks → Pile → Rooms

LVM: PV (disks) → VG (pool) → LV (volumes).

#11
Lazy Iguanas Gulp Ice-cream...

Link, IP, Gateway, Internet, DNS, Port, App. Ping ladder.

#12
Red Snails Drive Zippy Tractors

Process states: Running, Sleeping, Disk-wait, Zombie, sTopped.

#13
15 asks, 9 kills

SIGTERM asks nicely. SIGKILL is instant. 1 = reload, 2 = Ctrl+C.

#14
Nice = polite

nice 19 = most polite = least CPU. -20 = pushy.

#15
Small → big

cron: Minute, Hour, Day, Month, Weekday.

#16
start = now, enable = boot

enable --now does both. daemon-reload before restart.

#17
update = menu, upgrade = dinner

apt update refreshes the list. Nothing installs until upgrade.

#18
126 Run · 127 Find · 128+n

Not executable / not found / killed by signal n (137 = 9, 143 = 15).

#19
Three emergency brakes

set -e (error) -u (unset) -o pipefail (pipe). Scripts stop on trouble.

#20
Numbers = letters

-eq -lt for numbers, == != for strings.

#21
MERZ · 5-4-3-2

Redis 6379 spells MERZ on a keypad. Postgres 5432 counts down.

#22
-aG = Always Group

usermod -aG appends. Forget the a and groups get replaced.

#23
Ubuntu sudo, RHEL wheel

The admin group. Big wheel = boss.

#24
The t in /tmp

Sticky: Tenants can't remove each other's files.

#25
Minus = within, plus = older

find -mtime -1 is the last day. +7 is older than a week.

#26
> file 2>&1

Order matters. Redirect 1 first, then copy 2 to it.

#27
Cache is cash

buff/cache is spent instantly when apps need it. Read available.



Save it. Print it. Tape it next to your monitor.

Built by Shaam Natheshwar · DevOps / Cloud Engineer

Linux · AWS · Kubernetes · Terraform · Ansible · Jenkins · Monitoring